APB Slave Verification Environment Specification

Team

- Vishwa (Lead Verification Engineer)
- Praveen
- Hema

Objective

Develop a SystemVerilog class-based verification environment for the APB Slave DUT.

The verification architecture shall include:

1. Transaction Class
2. Generator
3. Driver
4. Monitor
5. Scoreboard
6. Environment
7. Test
8. Interface
9. Top Module

Mailbox-based communication shall be used between components.

DUT Interface

```
input  logic        PCLK;
input  logic        PRESETn;

input  logic        PSEL;
input  logic        PENABLE;
input  logic        PWRITE;

input  logic [31:0] PADDR;
input  logic [31:0] PWDATA;

output logic [31:0] PRDATA;
```

```
output logic        PREADY;
output logic        PSLVERR;
```

Transaction Class Requirements

Transaction class shall contain:

```
rand bit [31:0] PADDR;
rand bit [31:0] PWDATA;

bit [31:0] PRDATA;

bit PWRITE;
bit PSLVERR;
```

Maintain:

```
static bit [31:0] prev_addr;
static bit [31:0] last_valid_addr;
```

Write Transaction Constraints

Data

PWDATA shall be randomized.

```
PWDATA.rand_mode(1);
```

Address

Address shall be randomized with constraint:

```
PADDR = ++prev_addr;
```

This creates ascending write addresses.

After every successful randomization:

```
prev_addr = PADDR;
```

Error Learning Mechanism

Initially:

```
address_limit_constraint.constraint_mode(0);
```

No address upper limit is applied.

Generator continues generating ascending addresses.

Whenever DUT asserts:

```
PSLVERR == 1
```

Monitor shall capture the address causing the error.

That address shall be assumed as the first invalid address.

Example:

```
Valid region ends at 0x0000003C

Generator produces 0x00000040

PSLVERR = 1
```

Store:

```
last_valid_addr = 0x0000003C;
```

or

```
last_invalid_addr = 0x00000040;
```

(depending on implementation choice)

Then:

```
address_limit_constraint.constraint_mode(1);
```

Constraint becomes:

```
PADDR <= last_valid_addr;
```

From now onward all generated addresses remain within valid storage range.

This allows the testbench to automatically discover memory depth.

APB Protocol Compliance

Driver shall start a transaction only when:

```
PRESETn = 1
```

Write sequence:

Setup phase:

```
PSEL    = 1;
PENABLE = 0;
PWRITE  = 1;
```

Access phase:

```
PENABLE = 1;
```

Wait for:

```
PREADY == 1;
```

before completing transaction.

Read Mode Requirements

After write phase completion:

Enable read test.

Address ordering constraint:

```
PADDR > prev_addr
```

shall be disabled.

Read addresses must now be randomized within valid address range.

Example:

```
ascending_addr_constraint.constraint_mode(0);
valid_addr_constraint.constraint_mode(1);
```

Read accesses shall be performed using random valid addresses.

Scoreboard Requirements

Scoreboard shall maintain a reference model using associative array.

Example:

```
bit [31:0] mem_ref [bit[31:0]];
```

Generator → Scoreboard Communication

Whenever generator creates a WRITE transaction:

Address and data shall also be sent to scoreboard.

Example conceptually:

```
mem_ref[PADDR] = PWDATA;
```

This creates a golden reference memory.

Read Verification

Monitor captures:

```
PADDR
PRDATA
```

Scoreboard checks:

```
expected = mem_ref[PADDR];
actual   = PRDATA;
```

Pass:

```
expected == actual
```

Fail:

```
expected != actual
```

Report mismatch with:

- Address
- Expected data
- Actual data

Monitor Responsibilities

Capture:

```
PADDR
PWDATA
PRDATA
PWRITE
PSLVERR
```

Monitor shall publish transactions to scoreboard mailbox.

If:

```
PSLVERR == 1
```

monitor shall notify generator/environment about offending address.

Environment

Environment shall instantiate:

```
Generator
Driver
Monitor
Scoreboard
```

and connect all mailboxes.

Required communication paths:

```
Generator -> Driver

Monitor -> Scoreboard

Generator -> Scoreboard

Monitor -> Environment
```

The Environment shall also support feedback from error detection to generator for updating address constraints.

Coverage Goals

Minimum tests:

1. Reset Test
2. Sequential Write Test
3. Sequential Read Test
4. Random Read Test
5. Invalid Address Test
6. PSLVERR Detection Test
7. Boundary Address Test
8. Full Memory Readback Test

Deliverables

1. interface.sv
2. transaction.sv
3. generator.sv
4. driver.sv
5. monitor.sv
6. scoreboard.sv
7. environment.sv
8. test.sv
9. top.sv
10. Simulation Waveforms
11. Verification Report

Code should be modular and reusable so that additional APB slave features can be added later without major redesign.